1. Отступ Табуляция

· Хорошо: Использование табуляции, из-за которой код в разных IDE выглядит по-разному.
· Плохо: Код выровнен двумя пробелами, что гарантирует одинаковый вид у всех разработчиков.

2. Открывающая фигурная скобка на той же строке.

· Плохо: Скобка переносится на новую строку, что занимает лишнее место и нарушает визуальную связь с объявлением функции.
```c++
int main()
{
	return 0;
}
```
· Хорошо: Скобка остается в конце строки объявления.
```c++
int main() {
	return 0;
}
```

3. Фигурные скобки ставятся всегда, даже для одной строки.

· Плохо: Пропуск скобок, когда кто-то случайно добавляет вторую строку в блок if, и она выполняется всегда.
```c++
if (isReady)
	DoSomething();
	DoAnotherThing(); // Эта строка выполняется всегда, а не по условию!
```
· Хорошо: Скобки защищают код от ошибок при редактировании.
```c++
if (isReady) {
	DoSomething();
	DoAnotherThing();
}
```

4. После if, for, while, switch ставится пробел.

· Плохо: if(condition) — ключевое слово сливается со скобкой.

· Хорошо: if (condition) — пробел отделяет управляющую конструкцию от условия, улучшая читаемость.

5. Внутри круглых скобок пробелы ставятся.

· Плохо: if (x>y&&y<z) — выражение сжато, его трудно сканировать глазами.
· Хорошо: if (x>y&&y<z) — пробелы вокруг операторов делают логику выражения прозрачной.

6. Вокруг операторов =, +, -, == ставятся пробелы.

· Плохо: int a=b+c; — операторы прилипают к переменным, замедляя чтение.
· Хорошо: int a = b + c; — визуальное разделение операндов и операторов.

7. Классы и типы пишутся в PascalCase (с большой).

· Плохо: class playermanager — неясно, где класс, а где переменная.
· Хорошо: class PlayerManager — заглавные буквы сразу сигнализируют, что это тип данных.

8. Функции пишутся в PascalCase (тоже с большой).

· Плохо: void getPlayerScore(); — визуально не отличается от переменной.
· Хорошо: void GetPlayerScore(); — единый стиль для всех функций проекта.

9. Переменные пишутся в pascalCase (с маленькой).

· Плохо: int PlayerScore; — выглядит как класс или функция, вводит в заблуждение.
· Хорошо: int playerScore; — сразу понятно, что это переменная, а не тип.

10. Приватные поля класса пишутся в _pascalCase с подчёркиванием в конце.

· Плохо: private: int score; — внутри метода непонятно, работаешь ты с локальной переменной или с полем класса.
· Хорошо: private: int score_; — подчеркивание однозначно говорит: «это поле класса».

11. Константы пишутся с префиксом k, например kMaxSize.

· Плохо: const int maxSize = 100; — в кете легко перепутать с обычной переменной.
· Хорошо: const int kMaxSize = 100; — префикс 'k' мгновенно идентифицирует константу.

12. Вместо NULL используется nullptr.

· Плохо: MyClass* obj = NULL; — в перегруженных функциях NULL может восприниматься как целое число (0), а не как указатель.
· Хорошо: MyClass* obj = nullptr; — строго типизированное нулевое значение именно для указателей.

13. В заголовочных файлах обязательно используется include guard.

· Плохо: Отсутствие защиты приводит к ошибкам переопределения классов при множественном включении файла.
· Хорошо: #pragma once (или классические #define guards) гарантирует, что заголовочный файл подключится только один раз.

14. В классе сначала public, затем protected, затем private.

· Плохо: Когда приватные поля объявлены в самом верху, читатель видит реализацию до того, как поймет, как классом пользоваться.
· Хорошо: Сначала открытый интерфейс (public), с которым будут работать другие разработчики, затем защищенный (protected), и только в конце — детали реализации (private).

15. Указатель пишется как int* ptr.

· Плохо: int *ptr; — создает иллюзию, что звездочка относится к имени переменной, а не к типу.
· Хорошо: int* ptr; — подчеркивает, что int* — это единый тип (указатель на целое число).

16. При переопределении виртуальной функции используется override.

· Плохо: void Update() {} — компилятор не проверит, действительно ли ты переопределяешь виртуальную функцию родителя.
· Хорошо: void Update() override {} — ключевое слово заставляет компилятор проверить сигнатуру функции в базовом классе, защищая от опечаток.

17. Функции не должны быть слишком длинными (25 строк).

· Плохо: Функция на 200 строк, которая делает десять разных дел. Ее невозможно протестировать и отладить.
· Хорошо: Функция помещается на один экран (примерно 25 строк) и решает одну конкретную задачу.

18. Глобальные переменные не рекомендуется использовать.

· Плохо: Глобальная переменная gGameState, которая меняется в десяти разных файлах — уследить за багами невозможно.
· Хорошо: Состояние инкапсулировано внутри классов и передается явно через параметры функций.

19. Код должен быть максимально читаемым и понятным.

· Плохо: if (a&b) { c = d / e; } — магические операции без контекста.
· Хорошо: Самодокументируемый код, где имена переменных говорят сами за себя.

```c++
if (playerHasFlag & kCanJump) {
	verticalSpeed = gravity / jumpDivider;
}
```

20. Длина строки - 100 символов.

· Плохо: Строка растянута на 200 символов, и чтобы ее прочитать, нужно пользоваться горизонтальной прокруткой.
· Хорошо: Строка ограничена 100 символами, что позволяет удобно размещать редактор рядом с другими окнами (например, с браузером).